Super-Convergence

Результат: выполнены девять обучений CIFAR-10 / ResNet-56 на Kaggle: три режима для seeds 42, 43, 44. 1cycle получил **89.59 ± 0.24%** test accuracy за **17.00 ± 1.32 мин**; длинный multistep - **89.87 ± 0.25%** за **84.95 ± 6.32 мин**. После знака ± указано стандартное отклонение: оно показывает разброс результатов трёх запусков. Например, 89.59 ± 0.24% означает среднюю точность 89.59% и стандартное отклонение 0.24 процентного пункта.

1 Статья и исследовательский вопрос

Leslie N. Smith и Nicholay Topin, *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*, arXiv:1708.07120v3, 2018 [1]. Проверяем, может ли один цикл большого learning rate дать близкое конечное качество за меньшее число шагов и меньшее время обучения.

2 Настройки выполненного запуска

Таблица 1: Настройки основных запусков

Компонент	**Настройка всех основных запусков**
Данные	CIFAR-10: 45 000 train / 5 000 validation / 10 000 test
Разделение	По 500 изображений каждого класса в validation.
Модель	ResNet-56, post-activation, projection shortcuts, каналы 16/32/64; 855 770 параметров
Train augmentation	RandomCrop(32, padding=4), RandomHorizontalFlip; нормализация CIFAR-10
Оптимизация	SGD, FP32, batch 512, один GPU cuda:0, без накопления градиентов
Общие настройки	BatchNorm momentum 0.1; weight decay 0.0001 на всех параметрах; 2 data workers
Парные повторы	Seeds 42, 43, 44; одинаковая инициализация в паре методов
Окружение	Tesla T4; Python 3.13.15; PyTorch 2.11.0+cu128; torchvision 0.26.0+cu128; NumPy 2.1.3; CUDA 12.8; cuDNN 91900

Таблица 2: Режимы обучения

Метод	**Эпохи / шаги**	**LR и momentum**
baseline_long	120 / 10 560	LR 0.1; снижение ×0.1 на 50% и 75% шагов; momentum 0.9
baseline_short	24 / 2 112	То же снижение LR, но обучение длится 24 эпохи вместо 120
onecycle	24 / 2 112	Фазы 45%/45%/10%; LR 0.1/1.0/0.1/0.0001; momentum 0.95/0.85/0.95, затем 0.95

3 Что взято из статьи и что изменено

Таблица 3: Сопоставление настроек статьи и выполненного запуска

Компонент	**Основной пример статьи [1]**	**Выполненный запуск**
Реализация	Caffe, приложение A.4	PyTorch, одна Tesla T4
Train / validation	50 000 train, таблица 6	45 000 / 5 000; фиксированное разделение данных
Число шагов	Около 80 000 / 10 000 шагов, рисунок 1a	10 560 / 2 112 (120 / 24 эпохи)
Batch	1000; 512 также исследован в таблице 7	512
LR для CLR	0.1-3.0, таблица 6	0.1-1.0 по предварительному LR range test
Momentum	0.9, таблица 8	0.95-0.85-0.95 для 1cycle
1cycle	Один цикл и завершение с LR ниже начального, раздел 3	Фазы 45%/45%/10%; конец 0.0001
Выбор максимума LR	Пик test accuracy в LR range test, раздел 3	Validation; правило 80% и границы [0.2, 1.0]
Сравнение скорости	Шаги и test accuracy, рисунок 1a	Дополнительно время до 85% validation

CIFAR-10, ResNet-56, LR range test и идея 1cycle взяты из статьи. Разделение данных, seeds 42/43/44, число эпох 120/24, критерий 85%, выбор лучшего validation checkpoint и правило подбора LR заданы в выполненном эксперименте.

4 Измерения: точность и время

Таблица 4: Точность и время обучения по трём seeds

Метод	**Test accuracy, %** **Среднее ± SD**	**Время, мин** **Среднее ± SD**
baseline_long	89.87 ± 0.25	84.95 ± 6.32
baseline_short	84.00 ± 0.27	17.13 ± 1.44
onecycle	89.59 ± 0.24	17.00 ± 1.32

Таблица 5: Точность по отдельным seeds

Seed	**Long, %**	**Short, %**	**1cycle, %**	**Δ 1cycle-long, п.п.**
42	89.69	83.88	89.58	-0.11
43	90.16	84.31	89.83	-0.33
44	89.77	83.80	89.36	-0.41

Средняя парная разница 1cycle - long: **-0.28 ± 0.16 п.п.** Во всех трёх seeds 1cycle уступает на 0.11-0.41 п.п.

5 Измерения: время до 85% validation accuracy

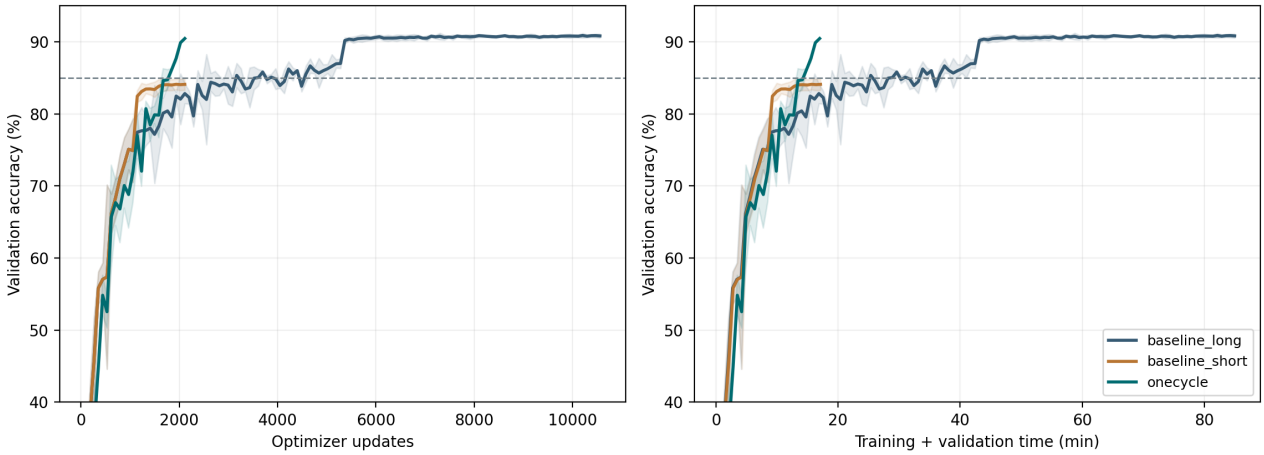


Рисунок 1. Линии показывают среднюю validation accuracy по трём запускам. Затенённая область вокруг каждой линии показывает среднее значение плюс-минус одно стандартное отклонение. На левом графике показано число шагов, на правом - среднее время обучения и validation. Пунктирная линия соответствует точности 85%.

Таблица 6: Время достижения 85% validation accuracy

Seed	**Long до 85%, мин / шаги**	**1cycle до 85%, мин / шаги**	**Отношение времени**
42	30.51 / 3520	16.20 / 1848	1.88
43	18.11 / 2376	12.89 / 1672	1.41
44	19.68 / 2552	12.84 / 1672	1.53

1cycle достигает 85% validation accuracy быстрее во всех трёх парах: среднее парное отношение времени **1.61 ± 0.25**, диапазон 1.41-1.88. Среднее время long: 22.76 ± 6.75 мин; 1cycle: 13.98 ± 1.93 мин.

По сохранённым историям validation accuracy короткий multistep достиг 85% только в запуске с seed 43: на последней, 24-й эпохе, через 16.29 мин и 2112 шагов. В запусках с seeds 42 и 44 точность не достигла 85% за 24 эпохи.

6 Предварительный LR range test и время обучения

Отдельный LR range test для seed 42: линейный рост 0.01 → 2.0 за 1000 обновлений, validation каждые 50 шагов. Пилот завершён без расходимости. Лучшее измерение: шаг 800, LR 1.6016, validation 76.54%. Итоговый максимум: **1.0**.

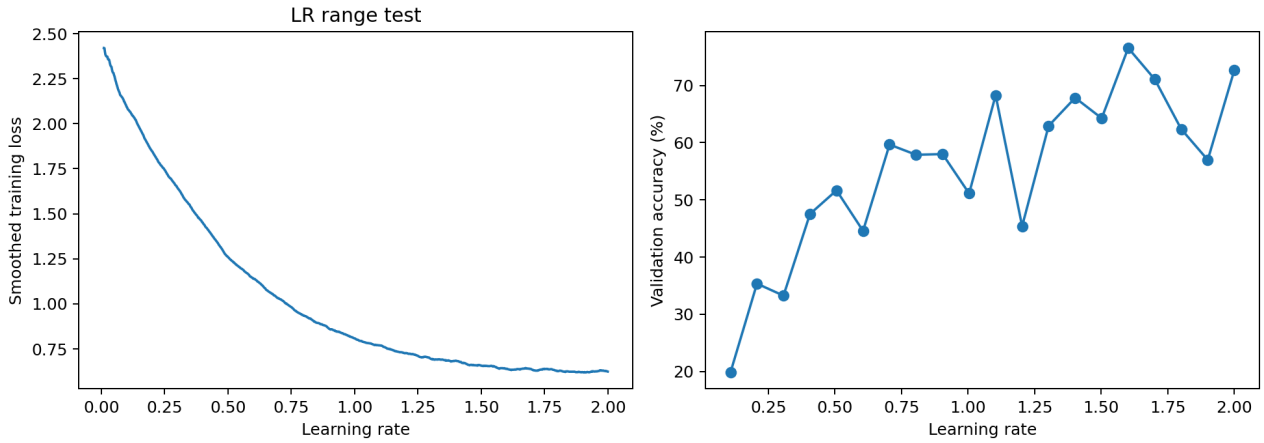


Рисунок 2. Предварительный LR range test для seed 42: зависимость training loss и validation accuracy от learning rate.

Пилот занял **8.03 мин**, не включённых в таблицу основного обучения. Суммарное время трёх 1cycle прогонов - 51.00 мин; с единственным пилотом - 59.04 мин. Для трёх длинных baseline прогонов - 254.84 мин.

7 Результат

Результаты выполненных запусков: 1cycle за 24 эпохи получил 89.59% test accuracy в среднем, длинный baseline - 89.87%, короткий - 84.00%. 1cycle раньше достигает выбранных 85% validation во всех трёх парах, но уступает long по test accuracy в каждой.

Полученные результаты показывают, что при 24 эпохах режим 1cycle обеспечивает более высокую точность, чем короткий baseline: преимущество составляет 5.59 процентного пункта. По сравнению с длинным baseline средняя точность ниже на 0.28 процентного пункта, а полный прогон занимает 17.00 мин вместо 84.95 мин. Таким образом, в выполненных экспериментах 1cycle позволил получить близкую к длинному baseline точность за существенно более короткий прогон. Это делает его полезным вариантом при ограниченном времени обучения.

Источники

[1] Smith & Topin, Super-Convergence, arXiv:1708.07120v3, 2018

[2] Авторский Caffe-репозиторий, зафиксированная версия

[3] CIFAR-10 Python, Kaggle Input

[4] Kaggle, seed 42

[5] Kaggle, seeds 43 и 44